

Détection du Port du Masque par Réseau de Neurones Convolutif (CNN)

Projet réalisé par : Omar Zouhairi et Yassine Sekkour
Matière : Deep Learning
Classe : 4IASD1

30 avril 2026

Table des matières

1	Introduction	3
2	Contexte et objectifs	3
3	Dataset utilisé	3
4	Prétraitement des images	3
4.1	Redimensionnement	4
4.2	Normalisation	4
4.3	Découpage train / validation / test	4
4.4	Data augmentation	4
5	Architecture du CNN	4
5.1	Schéma global du réseau	5
5.2	Utilité des couches	5
5.3	Optimisation et entraînement	5
6	Métriques d'évaluation	5
7	Résultats obtenus	6
8	Intégration dans le portfolio	6
9	Discussion	7
10	Conclusion générale	8

1 Introduction

Ce projet d'intelligence artificielle vise à détecter automatiquement si une personne porte un masque facial (*with_mask*) ou non (*without_mask*) à partir d'une image. Le modèle utilisé est un Réseau de Neurones Convolutif (CNN) construit *from scratch*, sans recours à une architecture pré-entraînée.

L'objectif principal est de proposer une démonstration complète, allant du traitement des données à l'intégration d'une interface de test via une API locale FastAPI. Le projet a été pensé pour être exploitable dans un portfolio, avec une inférence possible en temps réel depuis une webcam.

Conclusion de l'introduction. Le projet constitue une chaîne complète de classification d'images, depuis le dataset jusqu'à la démonstration interactive.

2 Contexte et objectifs

La problématique est simple à formuler : reconnaître la présence d'un masque facial dans un visage détecté par une caméra ou une image importée. Ce type d'application a un intérêt pédagogique fort, car il mobilise à la fois la vision par ordinateur, l'apprentissage profond, l'évaluation de modèles et le déploiement applicatif.

Les objectifs retenus sont les suivants :

- construire un CNN simple et interprétable ;
- entraîner le modèle sur un dataset public équilibré ;
- évaluer les performances sur un jeu de test inédit ;
- intégrer le modèle dans une API locale ;
- proposer une démonstration webcam depuis l'interface portfolio.

Conclusion de la partie. Le projet ne se limite pas à l'entraînement d'un modèle ; il illustre aussi la mise en production légère d'une solution de deep learning.

3 Dataset utilisé

Le dataset utilisé provient de Kaggle : *Face Mask Dataset* publié par *omkargurav*. Il contient au total 7 553 images réparties entre deux classes :

- 3 725 images *with_mask* ;
- 3 828 images *without_mask*.

Le dataset est suffisamment équilibré pour permettre un apprentissage fiable sans biais de classe majeur. Les images présentent des visages dans des conditions variées de pose, luminosité et cadrage.

Conclusion de la partie. La taille du jeu de données et son équilibre constituent une base solide pour une classification binaire robuste.

4 Prétraitement des images

Avant l'entraînement, plusieurs opérations ont été appliquées aux images brutes.

4.1 Redimensionnement

Toutes les images ont été converties en RGB puis redimensionnées en 128×128 pixels. Ce choix réduit la complexité calculatoire tout en conservant suffisamment de détail pour une tâche binaire.

4.2 Normalisation

Les pixels ont été normalisés dans l'intervalle $[0, 1]$ afin d'améliorer la stabilité de l'entraînement et la convergence du réseau.

4.3 Découpage train / validation / test

Le dataset a été séparé en trois sous-ensembles :

- entraînement ;
- validation ;
- test.

Le test set contient 1 133 images inédites, utilisées uniquement pour l'évaluation finale.

4.4 Data augmentation

Des transformations aléatoires ont été utilisées sur les données d'entraînement afin de réduire le surapprentissage et d'améliorer la robustesse du modèle face à des variations de pose, de luminosité ou d'orientation.

Espace réservé pour une capture du pipeline de prétraitement / augmentation

Conclusion de la partie. Le prétraitement vise surtout à rendre les entrées homogènes et à augmenter la généralisation du réseau.

5 Architecture du CNN

Le modèle est un CNN séquentiel construit manuellement. Il repose sur trois blocs convolutifs, suivis d'une partie entièrement connectée pour la classification finale.

5.1 Schéma global du réseau

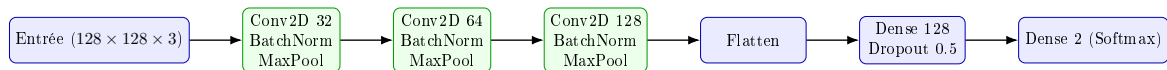


FIGURE 1 – Architecture du CNN utilisée pour la classification binaire.

5.2 Utilité des couches

- **Conv2D** : extrait des motifs visuels locaux, comme les contours, les textures et les formes liées au visage.
- **BatchNormalization** : stabilise les activations et accélère la convergence.
- **MaxPooling2D** : réduit la dimension spatiale et conserve les caractéristiques les plus discriminantes.
- **Flatten** : transforme la carte de caractéristiques en vecteur exploitable par les couches denses.
- **Dense 128** : combine les caractéristiques extraites pour apprendre une représentation de décision.
- **Dropout 0.5** : limite le surapprentissage en désactivant aléatoirement une partie des neurones pendant l'entraînement.
- **Dense 2 + Softmax** : produit la probabilité finale pour chaque classe.

5.3 Optimisation et entraînement

Le modèle a été compilé avec :

- **Optimiseur** : Adam ;
- **Fonction de perte** : categorical_crossentropy ;
- **Métrique** : accuracy.

Les callbacks suivants ont été utilisés :

- **EarlyStopping** pour arrêter l'entraînement si la validation n'améliore plus ;
- **ReduceLROnPlateau** pour réduire automatiquement le learning rate ;
- **ModelCheckpoint** pour sauvegarder le meilleur modèle sous `best_model.keras`.

Comme illustré dans la Figure ??, la chaîne de traitement suit une logique progressive : extraction locale des motifs, réduction spatiale, puis classification finale.

Conclusion de la partie. L'architecture reste volontairement simple, mais elle est suffisamment expressive pour apprendre les caractéristiques nécessaires à la tâche.

6 Métriques d'évaluation

L'évaluation du modèle a été faite avec des métriques complémentaires pour obtenir une vue fine de la performance.

- **Accuracy** : proportion globale de prédictions correctes.
- **Precision** : fiabilité des prédictions positives.
- **Recall** : capacité à retrouver toutes les instances d'une classe.
- **F1-Score** : compromis entre précision et rappel.
- **Matrice de confusion** : détail des vrais positifs, faux positifs, vrais négatifs et faux négatifs.

Conclusion de la partie. L'emploi de plusieurs métriques est indispensable pour juger correctement un classifieur binaire, surtout dans un contexte de sécurité sanitaire.

7 Résultats obtenus

L'évaluation finale sur le test set de 1 133 images inédites a donné les résultats suivants :

TABLE 1 – Synthèse des résultats globaux et par classe.

Métrique	Valeur	Interprétation
Accuracy globale	89,32%	Bon niveau global
F1-Score macro	89%	Équilibré entre les classes
Recall with_mask	99%	Excellente détection des masques
Recall without_mask	80%	Quelques faux négatifs
Precision with_mask	83%	Prédictions positives fiables
Precision without_mask	99%	Très peu de faux positifs

Le Tableau ?? résume les performances globales et met en évidence le bon comportement du modèle sur la classe portant un masque.

La matrice de confusion synthétise également les résultats :

- vrais négatifs (TN) : 542 ;
- vrais positifs (TP) : 536 ;
- faux positifs (FP) : 17 ;
- faux négatifs (FN) : 38.

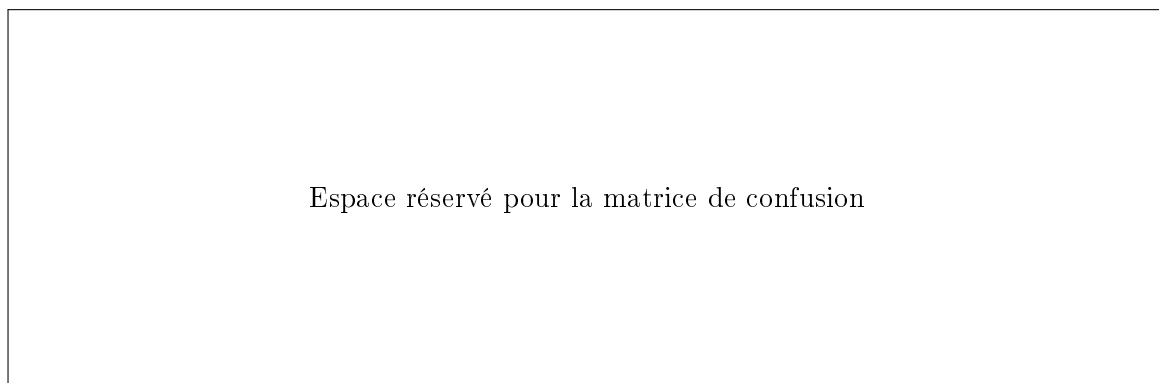


FIGURE 2 – Matrice de confusion du test set.



Espace réservé pour les courbes d'entraînement (loss / accuracy)

FIGURE 3 – Courbes d'apprentissage du modèle.

Les Figures ?? et ?? complètent le tableau en montrant respectivement la répartition des erreurs et l'évolution de l'apprentissage.

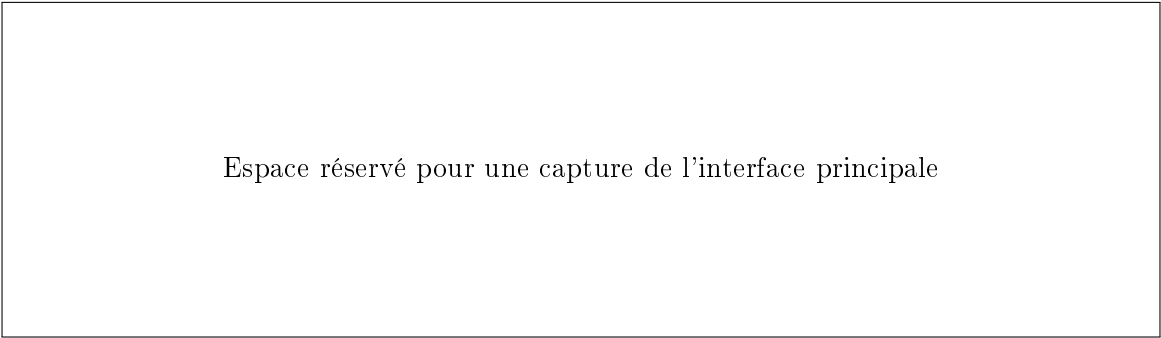
Conclusion de la partie. Les performances sont solides et cohérentes avec l'objectif du projet. Le modèle détecte très bien les visages portant un masque, ce qui est particulièrement intéressant pour une démonstration pratique et pour des scénarios où l'on souhaite limiter les faux positifs. La matrice de confusion montre également que les erreurs restent contenues et surtout localisées sur la classe sans masque, ce qui ouvre la porte à des améliorations ciblées sur ce point précis.

8 Intégration dans le portfolio

Le modèle a été intégré à une interface de démonstration reliée à une API locale FastAPI. Le flux utilisateur est simple :

- ouverture de l'interface depuis le portfolio ;
- sélection d'une image ou activation de la webcam ;
- envoi vers l'API locale ;
- retour de la prédiction en temps réel.

L'API expose un endpoint de prédiction et un endpoint de santé pour vérifier que le service est opérationnel. Cette couche d'intégration permet de montrer un cas d'usage concret et visualisable.



Espace réservé pour une capture de l'interface principale

FIGURE 4 – Interface principale de la démo image.

8.1 Démo webcam

La démonstration webcam est pensée comme une extension plus immersive de l'interface. Elle permet de tester le modèle en continu sur un flux vidéo et de visualiser le résultat en direct.

Comme aucune capture n'est disponible pour le moment, un espace est réservé ci-dessous pour insérer l'aperçu final une fois la démo stabilisée.

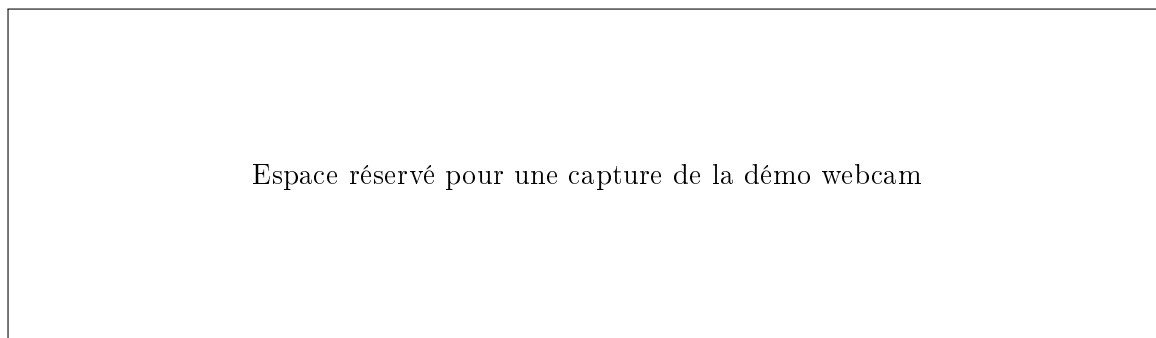


FIGURE 5 – Zone réservée pour la capture de la démo webcam.

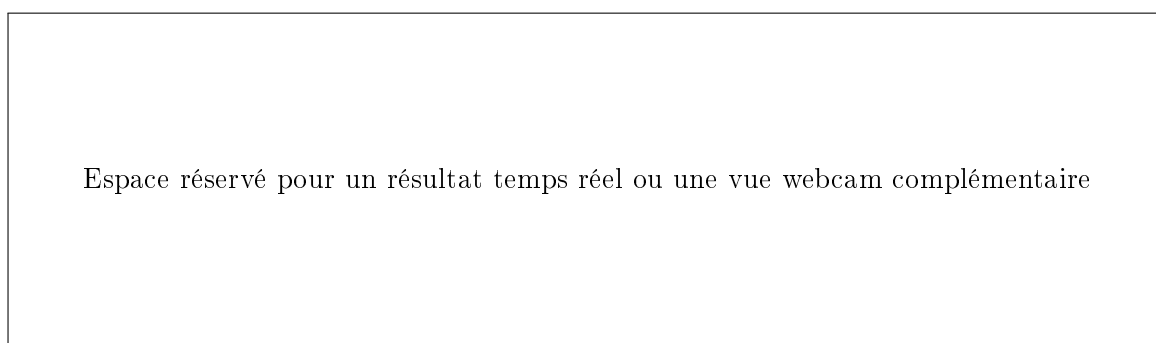


FIGURE 6 – Exemple de vue webcam ou résultat temps réel.

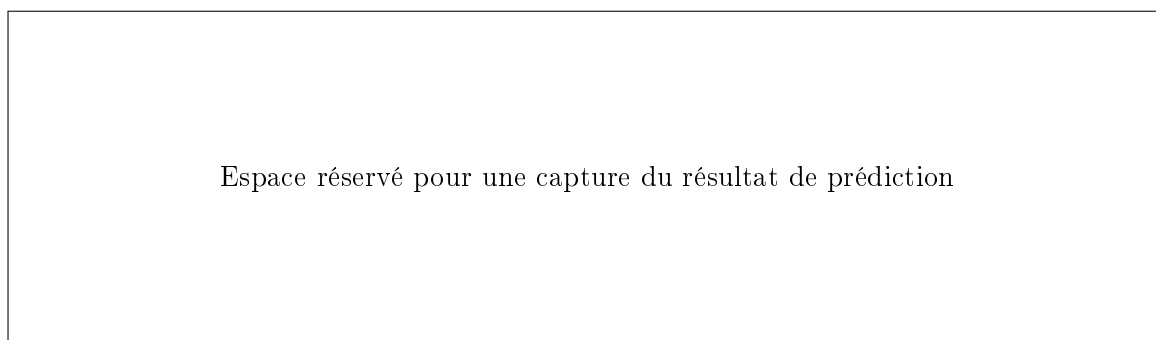


FIGURE 7 – Exemple de résultat de prédiction.

La Figure ?? pourra être remplacée par une capture finale du flux vidéo, tandis que les Figures ?? et ?? serviront à documenter l'interface et un cas de classification réussi.

Conclusion de la partie. L'intégration applicative transforme le modèle de classification en une démonstration exploitable et présentable dans un portfolio. L'ajout d'une interface webcam renforce la valeur pédagogique du projet, car il rapproche la solution d'un usage réel et montre comment un modèle de vision peut être intégré dans une application web locale. Cette partie illustre aussi l'intérêt d'un chaînage propre entre entraînement, API et interface utilisateur, ce qui est essentiel pour passer d'un prototype à une démonstration crédible.

9 Discussion

Le modèle répond correctement à la tâche principale, mais quelques limites restent à noter :

- sensibilité à la qualité du cadrage et à la luminosité ;
- confusion possible dans des scènes webcam trop bruitées ;
- dépendance à un bon recadrage du visage pour la démo temps réel.

Pour améliorer encore le système, on pourrait :

- intégrer une détection de visage explicite avant la classification ;
- enrichir la data augmentation ;
- tester des architectures plus profondes ou hybrides ;
- appliquer une calibration du seuil de décision.

Conclusion de la partie. Le modèle est déjà exploitable, mais une version plus robuste gagnerait à séparer détection de visage et classification. Cette évolution améliorerait le cadrage des entrées et réduirait les erreurs observées sur des scènes webcam moins contrôlées.

10 Conclusion générale

Ce projet a permis de réaliser une chaîne complète de deep learning, depuis la préparation du dataset jusqu'à l'intégration d'une interface de démonstration en local. Le CNN construit from scratch atteint des performances solides et constitue une bonne base pédagogique pour comprendre le fonctionnement d'un classifieur d'images.

Au-delà des résultats numériques, le projet montre aussi la capacité à industrialiser légèrement une solution IA : entraînement, export du modèle, API locale, intégration frontend et démonstration webcam. Cela en fait une réalisation cohérente pour un portfolio académique et technique.

Conclusion générale. Ce projet est à la fois une preuve de compétence en deep learning et une base réutilisable pour une future version plus robuste ou plus précise. Il montre la capacité à structurer un pipeline complet, à documenter les choix techniques et à transformer un modèle entraîné en démonstrateur concret. En pratique, le projet peut être enrichi par une meilleure détection préalable du visage, une interface webcam plus fluide et une optimisation plus poussée de la séparation des classes.

Annexes

Espace réservé pour les captures d'écran supplémentaires :

- capture de la page d'accueil du portfolio ;
- capture de la fiche projet ;
- capture de l'interface de démo image ;
- capture de l'interface de démo webcam ;
- capture d'une prédiction réussie ;
- capture de l'API en fonctionnement.

Fin du rapport.